Server Maintenance – Step-by-Step Documentation

1. Purpose

The purpose of this document is to provide a standard procedure for performing server maintenance safely.

The maintenance includes:

- Checking server health and capacity
- Checking CPU, memory, swap, and storage
- Checking Docker containers and images
- Performing OS/package patch updates
- Rebooting the server when required
- Verifying the server after reboot
- Starting required Docker containers
- Performing Docker cleanup only when it is safe
- Recording maintenance results

---

2. Pre-Maintenance Checks

Before starting maintenance, confirm that:

- The server is accessible through SSH/PuTTY.
- The maintenance window is approved.
- Users/teams are informed if applications will be unavailable.
- Important containers and services are identified.
- No critical deployment or scraper activity is running.

---

3. Check Server Uptime

Run:

```
uptime
```

Example:

```
up 27 days, 21:04
load average: 0.55, 0.60, 0.61
```

Purpose

This shows:

- How long the server has been running
- Number of logged-in users
- Load average

Record the uptime before maintenance.

---

4. Check CPU and Memory Usage

Run:

```
top
```

The important section is:

```
%Cpu(s): ...
MiB Mem : ...
MiB Swap: ...
```

CPU Usage

For example:

```
%Cpu(s): 1.8 us, 1.7 sy, 96.3 id
```

CPU usage can be calculated as:

```
User + System CPU
= 1.8% + 1.7%
= 3.5%
```

Memory Usage

For example:

```
Mem: 15993.2 total
     4184.6 used
```

Memory usage:

```
4184.6 / 15993.2 × 100
= 26.2%
```

Swap

Check:

```
Swap: 2048.0 total
       2047.6 used
```

This means approximately 100% of swap is being used.

Record CPU, memory, swap, and load average in the maintenance report.

---

5. Check Storage Usage

Run:

```
df -h
```

For the main root filesystem, normally check `/`.

Example:

```
/dev/sda3    590G    267G    293G    48%    /
```

Record:

```
Storage Total: 590 GB
Storage Used:  267 GB
Storage Usage: 48%
```

Important

Do not automatically consider every mounted filesystem as the server's main storage.

For example, if there is a separate mount such as:

```
/mnt/wiley_laps
```

and it is intentionally excluded from the maintenance report, use the root `/` filesystem for the capacity calculation.

---

6. Check Docker Status

Check running containers:

```
docker ps
```

Check all containers, including stopped/exited containers:

```
docker ps -a
```

Why check `docker ps -a`?

It helps identify:

- Running containers
- Stopped containers
- Exited containers
- Restarting containers

Pay special attention to containers showing:

```
Restarting
Exited
```

These may require investigation.

---

7. Check Docker Storage

Run:

```
docker system df
```

Example:

```
TYPE            TOTAL     ACTIVE    SIZE      RECLAIMABLE
Images          344       43        152.7GB   149.4GB (97%)
Containers       44       30         11.68GB   549.5MB (4%)
```

```
Local Volumes     3         1         1.293MB   242.4kB
Build Cache       0         0         0B
```

This shows:

- Number of Docker images
- Active images
- Image storage
- Reclaimable image storage
- Number of containers
- Container storage
- Volumes
- Build cache

Important

`RECLAIMABLE` does **not** mean that space has already been removed.

For example:

```
149.4GB (97%)
```

means Docker believes that amount can potentially be reclaimed.

It does not mean 149.4 GB was deleted.

---

8. Decide Whether Docker Cleanup Is Safe

Do not automatically run:

```
docker system prune
```

on a production server.

First check:

```
docker ps -a
```

If there are exited containers, determine whether they are still required.

If exited containers may be required

Do not run:

```
docker system prune
```

Keep them until they are confirmed as unnecessary.

If cleanup is approved

A safer cleanup can be performed after confirming what can be removed.

For example:

```
docker image prune -f
```

This removes dangling images.

For broader cleanup, use `docker system prune` only when the team has confirmed that unused containers/images can be removed.

---

9. Check OS Information

Run:

```
cat /etc/os-release
```

Record:

- OS Name
- OS Version

Example:

```
NAME="Ubuntu"
VERSION_ID="22.04"
VERSION="22.04.5 LTS (Jammy Jellyfish)"
```

Record:

```
OS Name: Ubuntu
OS Version: 22.04.5 LTS
```

---

10. Check Kernel / Patch Level

Run:

```
uname -r
```

Example:

```
6.8.0-136-generic
```

Record this as the **Patch Level / Kernel Version**.

---

11. Check Packages That Can Be Updated

Before patching, check:

```
apt list --upgradable 2>/dev/null
```

To count them:

```
apt list --upgradable 2>/dev/null | tail -n +2 | wc -l
```

This gives an indication of how many packages are currently available for upgrade.

---

12. Perform OS Package Updates

When the maintenance window is approved, run:

```
sudo apt update
sudo apt upgrade -y
sudo apt full-upgrade -y
```

Explanation

`sudo apt update`

Refreshes the package repository information.

It does not normally install updates.

`sudo apt upgrade -y`

Installs available package updates.

`sudo apt full-upgrade -y`

Performs a more complete upgrade and can handle dependency changes that may require installing or removing packages.

---

13. Check Whether Reboot Is Required

Run:

```
if [ -f /var/run/reboot-required ]; then echo "Yes"; else echo "No"; fi
```

If it returns:

```
Yes
```

a reboot is recommended/required.

---

14. Before Reboot – Check Containers

Run:

```
docker ps
```

Record which containers are currently running.

Important

You do not normally need to manually stop every Docker container before a planned server reboot.

A server reboot will shut down the running containers as part of the system shutdown.

If your maintenance procedure specifically requires manually stopping them, use:

```
docker stop $(docker ps -q)
```

This stops all currently running containers but does not delete them.

---

15. Reboot the Server

After confirming the maintenance window:

```
sudo reboot
```

The SSH connection will disconnect.

Wait for the server to complete its boot process.

---

16. Verify Server After Reboot

Reconnect through SSH and run:

```
uptime
```

Confirm that the uptime shows a recent boot.

Example:

```
up 12 minutes
```

Then check:

```
top
```

Verify:

- CPU usage
- Memory usage
- Swap
- Load average

Then check storage:

```
df -h
```

---

17. Check Docker After Reboot

Run:

```
systemctl status docker --no-pager
```

Confirm Docker is active.

Then:

```
docker ps -a
```

Check whether the required containers are running.

---

18. Start Required Containers

Some containers may automatically start after reboot depending on their Docker restart policy.

If a required container is stopped, start it individually:

```
docker start <container_name>
```

Example:

```
docker start apex-api
```

Important

Avoid blindly running:

```
docker start $(docker ps -aq)
```

on servers containing old/exited containers.

That command starts **every container**, including containers that may have intentionally been left stopped.

Start only the containers that are actually required.

---

19. Verify Container Health

Run:

```
docker ps
```

Confirm required containers show:

```
Up
```

If a container immediately exits or shows `Restarting`, check its logs:

```
docker logs --tail 100 <container_name>
```

For example:

```
docker logs --tail 100 ai-dashboard-container
```

---

20. Check for Container Restart Problems

If a container shows:

```
Exited (137)
```

check whether it was killed because of an out-of-memory condition:

```
docker inspect <container_name> --format '{{.State.OOMKilled}}'
```

If the result is:

```
true
```

the container was killed because of an out-of-memory condition.

---

21. Post-Maintenance Validation

After reboot, verify:

Server

```
uptime
```

CPU / Memory

```
top
```

Storage

```
df -h
```

Docker

```
docker ps
```

All containers

```
docker ps -a
```

Docker storage

```
docker system df
```

Confirm that the server and required applications are operational.

---

22. Docker Image Review

Check images:

```
docker images
```

Or:

```
docker system df
```

Record:

- Total images checked
- Outdated images, if verified

- Images rebuilt
- Images removed
- Registry used

Do not claim an image is outdated unless it was actually checked against the appropriate registry/tag/digest.

If no rebuild was required:

```
Images Rebuilt: 0
Rebuild Reason: N/A – No image rebuild was required.
```

---

23. Docker Cleanup

If cleanup is part of the scheduled maintenance and unused resources have been confirmed safe:

```
docker system prune
```

Before using it, remember that it can remove unused Docker resources, including stopped containers.

If there are exited containers that may still be needed, **do not run the prune command until they are confirmed safe to remove**.

After cleanup, check:

```
docker system df
```

Record the actual:

```
Space Freed
```

from Docker's prune output.

Do not confuse **reclaimable space** with **space actually freed**.

---

24. Maintenance Reporting

After completing the maintenance, record the following.

Server Capacity

```
Result:
CPU Avg %:
Memory Avg %:
Storage Used %:
Storage Used GB:
Storage Total GB:
Trend:
Recommendation:
Remarks:
```

OS Patch

```
Result:
OS Name:
OS Version Before:
OS Version After:
Packages Updated:
Patch Level:
Reboot Required:
Reboot Done:
Issues Found:
Rollback Needed:
Remarks:
```

Docker Image Review

```
Result:
Total Images Checked:
Outdated Images:
Images Rebuilt:
Images Removed:
Container Registry:
Rebuild Reason:
Remarks:
```

Docker Cleanup

```
Result:
Images Removed:
Containers Removed:
Volumes Removed:
Space Freed GB:
```

```
Artifact Registry Cleaned:
Remarks:
```

---

25. Recommended Standard Flow

For normal server maintenance, follow this sequence:

```
1. Inform / confirm maintenance window
          ↓
2. SSH into server
          ↓
3. uptime
          ↓
4. top
          ↓
5. df -h
          ↓
6. docker ps
          ↓
7. docker ps -a
          ↓
8. docker system df
          ↓
9. Check OS version
          ↓
10. Check kernel version
          ↓
11. Check available package updates
          ↓
12. sudo apt update
          ↓
13. sudo apt upgrade -y
          ↓
14. sudo apt full-upgrade -y
          ↓
15. Check reboot requirement
          ↓
16. Check Docker containers
          ↓
17. sudo reboot
          ↓
18. Reconnect after reboot
          ↓
19. uptime
          ↓
20. top
          ↓
```

```
21. df -h
          ↓
22. systemctl status docker
          ↓
23. docker ps -a
          ↓
24. Start required stopped containers
          ↓
25. Verify application/container health
          ↓
26. Perform Docker cleanup only if approved
          ↓
27. docker system df
          ↓
28. Complete maintenance report
```

26. Important Safety Rules

Do not blindly use:

```
docker system prune
```

if the server has exited containers that may be required.

Do not blindly use:

```
docker start $(docker ps -aq)
```

because it starts every container, including old containers that may intentionally be stopped.

Do not repeatedly reboot

If a server becomes unreachable or enters **Emergency Mode** after reboot, stop further reboot attempts and contact the IT/infrastructure team.

If Emergency Mode occurs

Use the server console and investigate the actual boot error. Useful checks include:

```
systemctl --failed
```

```
journalctl -xb -p err
```

Do not immediately modify `/etc/fstab` or run filesystem repair commands without identifying the actual failure first.

27. Final Completion Checklist

Before marking maintenance as **Done**, confirm:

- [ ] Server accessible after reboot
- [ ] OS patching completed
- [ ] Kernel/patch level recorded
- [ ] CPU checked
- [ ] Memory checked
- [ ] Swap checked
- [ ] Storage checked
- [ ] Docker service running
- [ ] Required containers running
- [ ] No unexpected container restart/exit
- [ ] Docker cleanup completed only if approved
- [ ] Post-maintenance checks completed
- [ ] Maintenance documentation updated
- [ ] Any issues reported to the appropriate team

Final status should be marked `Done` only when the required maintenance activities and post-reboot validation have been completed successfully.